

SISTEMA IOT FULL-STACK PARA MONITORAMENTO E DETECÇÃO DE QUEDAS EM IDOSOS

Rafael Ryan Ramos de Souza, Rodrigo de Souza Scarinci

Universidade do Vale do Paraíba/Engenharia da Computação, Avenida Shishima Hifumi, 2911, Urbanova - 12244-000 - São José dos Campos-SP, Brasil, e-mail dos autores correspondente: souzaramos2001@gmail.com, rodrigo.scarinci@gmail.com

Resumo

Este trabalho apresenta o desenvolvimento de um sistema IoT full-stack para monitoramento e detecção experimental de quedas em idosos. A proposta integra um dispositivo baseado em ESP32 e sensor inercial MPU6050, comunicação MQTT, backend em Node.js com banco MySQL, atualização em tempo real por Socket.IO e dashboard web para acompanhamento de pacientes, dispositivos e alertas. A metodologia contemplou a organização da arquitetura em camadas, a implementação de uma máquina de estados para diferenciar movimento intenso, queda suspeita e queda confirmada, além da criação de um modo de demonstração com leitura do sensor a 25 ms e publicação de telemetria a 500 ms. Os resultados preliminares indicaram funcionamento ponta a ponta do protótipo, com telemetria exibida em 120 amostras, registro histórico, exportação de relatórios e confirmação de queda controlada por impacto, mudança de orientação e imobilidade. Conclui-se que o sistema atende ao objetivo acadêmico de demonstrar uma solução modular, auditável e expansível para apoio ao monitoramento remoto, embora não represente dispositivo médico validado nem substitua avaliação clínica.

Palavras-chave: Internet das Coisas. ESP32. Detecção de quedas. MQTT. Monitoramento remoto.

Área do Conhecimento: Engenharias / Engenharia de Computação.

Introdução

O envelhecimento populacional aumenta a demanda por soluções tecnológicas capazes de apoiar o cuidado, a autonomia e o acompanhamento remoto de pessoas idosas. Nesse contexto, quedas e longos períodos de imobilidade são eventos de risco que podem exigir resposta rápida de familiares ou cuidadores. Sistemas embarcados com sensores inerciais se mostram adequados para protótipos acadêmicos por permitirem observar aceleração, variação angular e mudanças de postura com baixo custo e boa disponibilidade de componentes.

A Internet das Coisas permite conectar o dispositivo físico a uma aplicação de monitoramento, mantendo o processamento local para eventos críticos e utilizando a rede para registro, visualização e resposta operacional. No projeto desenvolvido, a detecção inicial ocorre no firmware, enquanto o backend recebe status, telemetria e eventos, persiste as informações e emite atualizações para o dashboard. Esse desenho evita que a interface web seja responsável por decidir uma queda, reduz dependência do navegador e permite preservar histórico para auditoria.

O objetivo deste trabalho é apresentar o desenvolvimento e a validação preliminar de um sistema IoT para monitoramento de quedas, integrando hardware, comunicação, backend, banco de dados e frontend web em uma solução demonstrável para disciplina de Projetos em Engenharia da Computação II.

Além da detecção embarcada, o projeto considera requisitos de rastreabilidade e organização dos dados. Por isso, a aplicação separa pacientes, dispositivos e alertas por organização, utiliza autenticação por token e registra histórico de eventos e ações humanas. Essa abordagem é relevante em sistemas assistivos, pois a interpretação de um alerta depende tanto do sinal físico capturado quanto do contexto operacional em que o dispositivo está vinculado.

Metodologia

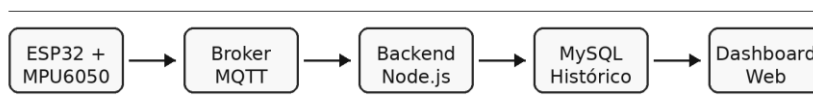
A solução foi organizada em camadas para separar aquisição física, comunicação, processamento, persistência e interface. No dispositivo, foi utilizado um ESP32 conectado a um sensor inercial MPU6050 para leitura de aceleração e giroscópio. O firmware realiza amostragem periódica, calcula grandezas como aceleração resultante, variação angular e orientação aproximada, e aplica uma máquina de estados baseada na sequência impacto, mudança de orientação e imobilidade. Eventos sem todos os critérios são classificados como movimento intenso ou queda suspeita, enquanto a queda confirmada exige impacto, rotação/postura compatível e repouso após o evento.

A Ciência do NANO e seu impacto transformador no MACRO

A comunicação entre o dispositivo e o servidor ocorre por MQTT, com tópicos separados para status, telemetria e eventos. O backend, implementado em Node.js, resolve a identidade do dispositivo, vincula dados à organização e ao paciente ativo, grava informações em MySQL e emite eventos para o frontend por Socket.IO. O painel web foi desenvolvido com React e Vite, permitindo login com JWT, escopo multi-tenant, visualização de dispositivos, pacientes, alertas, histórico e exportação JSON/PDF.

Para a apresentação acadêmica foi implementado o modo Demo apresentação. Nesse perfil, o sensor realiza leitura interna a cada 25 ms e a telemetria MQTT é publicada a cada 500 ms, enquanto o modo Normal preserva leitura a 50 ms e telemetria a 2000 ms. A interface exibe o modo ativo, a telemetria recente em até 120 amostras e o motivo técnico da decisão do detector. Também foi criada uma estimativa experimental de bateria por tempo, recalibrada manualmente pelo portal do ESP32, com taxa inicial de 33,5 min/% e ajuste gradual por calibrações sucessivas.

A validação foi realizada por meio de testes automatizados do backend, testes de integração com ingestão MQTT, compilação do firmware e testes manuais controlados com o hardware. Para evitar interpretações indevidas, os testes físicos foram executados apenas com a caixa do sensor em superfície macia, sem simular queda com pessoa. Foram observados o estado online do dispositivo, a atualização do gráfico, a criação de evento, a abertura de alerta e a persistência das evidências técnicas.



Socket.IO atualiza o painel em tempo real; eventos e telemetria ficam persistidos para auditoria.

Figura 1 - Arquitetura geral do sistema IoT de monitoramento de quedas.

Fonte: elaborado pelos autores.

Tabela 1 - Perfis de operação do protótipo.

Perfil	Leitura interna	Telemetria MQTT	Uso previsto
Normal	50 ms	2000 ms	Operação conservadora e menor carga
Demo apresentação	25 ms	500 ms	Demonstração em bancada com maior fluidez visual

Fonte: elaborado pelos autores.

Resultados

O sistema apresenta integração funcional entre o dispositivo físico, o broker MQTT, o backend, o banco de dados e o dashboard web. Durante a validação de bancada, o ESP32 permaneceu online, publicou telemetria em tempo real e permitiu a visualização de aceleração resultante no painel. A tela de detalhe do dispositivo apresentou 120 amostras recentes no modo Demo, além de diagnóstico de sensor, idade da amostra, RSSI, tópicos MQTT observados e modo do detector.

Em teste controlado, realizado com a caixinha contendo o sensor sobre superfície macia, foi registrada uma queda confirmada. O evento foi classificado a partir da combinação de pico de aceleração, variação angular e imobilidade posterior. De forma coerente com a máquina de estados definida no firmware, movimentos bruscos sem imobilidade foram tratados como movimento intenso ou queda suspeita, reduzindo a chance de acionar o buzzer para qualquer deslocamento comum.

No backend, foram preservados histórico de eventos, telemetria, alertas, ações humanas e auditoria. Também foram implementadas rotas e telas para desvincular paciente, desperear dispositivo para demonstração, arquivar paciente sem exclusão física e exportar relatórios. A Tabela 2 resume os principais resultados técnicos observados.

A interface web apresentou fluxo de autenticação, seleção de organização ativa, listagem de pacientes e dispositivos, detalhe técnico do sensor, alertas e histórico. A exportação em JSON e a visualização imprimível em PDF permitiram registrar os alertas filtrados e suas evidências. O portal de manutenção do ESP32 também se mostrou útil para ajustar Wi-Fi, MQTT, modo de operação, bateria manual e teste de buzzer sem interromper completamente a operação do dispositivo.

A Ciência do NANO e seu impacto transformador no MACRO

Tabela 2 - Principais resultados técnicos do protótipo.

Recurso validado	Resultado observado
Comunicação MQTT	Publicação de status, telemetria e eventos
Dashboard	Atualização por Socket.IO e gráfico com 120 amostras
Deteção de queda	Queda confirmada em teste controlado sobre superfície macia
Alertas	Registro histórico e ações operacionais de atendimento
Bateria	Estimativa por calibração manual, ainda sem medição elétrica real

Fonte: elaborado pelos autores.

Discussão

Os resultados indicam que a divisão de responsabilidades entre firmware, backend e frontend é adequada ao escopo do projeto. A decisão inicial no dispositivo reduz a dependência da rede para acionar o alarme local, enquanto o backend centraliza persistência, escopo de organização, histórico e evidências. Essa separação também facilita a manutenção, pois alterações no dashboard não modificam diretamente a lógica de detecção embarcada.

O modo Demo foi importante para tornar a apresentação mais clara, porém deve ser interpretado como perfil experimental. A redução de limiares e o aumento da frequência de publicação ajudam na visualização e comportamento do sistema, mas podem aumentar falsos positivos e carga no broker, no backend e no banco de dados. Por esse motivo, o modo Normal permanece conservador, e a documentação informa que o protótipo não possui calibração clínica.

A estimativa de bateria por tempo também deve ser entendida como recurso operacional de demonstração. Seu cálculo depende de medição externa declarada, e o processo gradual de calibração manual do operador pode reduzir a defasagem da estimativa ao longo do tempo. O uso de um divisor resistivo calibrado ou de um circuito medidor dedicado, como fuel gauge, é uma evolução futura para obter uma leitura elétrica real.

Outro ponto relevante é a rastreabilidade. O protótipo não se limita a mostrar um número instantâneo do sensor, pois mantém histórico de telemetria, eventos, alertas e ações. Essa característica permite revisar o que ocorreu, quando ocorreu e qual decisão operacional foi tomada. Para uma evolução futura, esse histórico pode apoiar a criação de uma base de testes maior, com diferentes movimentos de bancada, intensidades e posições do sensor.

Apesar da integração funcional, a solução apresenta limitações. A detecção utiliza limiares e regras heurísticas, não um modelo validado clinicamente. Além disso, a posição de uso do sensor, a fixação no corpo, o tipo de superfície e a intensidade do impacto podem alterar significativamente os sinais obtidos. Portanto, antes de qualquer aplicação real, seriam necessários estudos com protocolo adequado, autorização ética quando aplicável e comparação com métodos reconhecidos de avaliação.

Conclusão

O trabalho apresentou um sistema IoT full-stack para monitoramento experimental de quedas em idosos, integrando ESP32, MPU6050, MQTT, backend, MySQL, Socket.IO e dashboard web. A implementação permitiu demonstrar telemetria em tempo real, detecção de queda por máquina de estados, registro de eventos e alertas, exportação operacional e recursos de administração acadêmica.

A validação preliminar mostrou que o protótipo consegue diferenciar movimentos intensos de queda confirmada quando há impacto, mudança de orientação e imobilidade posterior. Apesar dos resultados positivos, o sistema ainda deve ser tratado como protótipo acadêmico: não foi validado clinicamente, não deve ser testado com pessoas e requer calibração com maior base de dados antes de uso real. Como trabalhos futuros, destacam-se a ampliação dos testes, a medição automática de bateria, o refinamento das features de movimento e a avaliação controlada de falsos positivos.

Dessa forma, o principal resultado do projeto está na integração de uma arquitetura completa e auditável, capaz de demonstrar conceitos de IoT, sistemas embarcados, banco de dados, segurança de API e interface em tempo real. O protótipo pode servir como base para melhorias técnicas e para discussões acadêmicas sobre confiabilidade, ética, usabilidade e segurança em sistemas de monitoramento remoto.

Referências

ESPRESSIF SYSTEMS. ESP32 Series Datasheet. Disponível em:

https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf. Acesso em: 09 jun. 2026.

OASIS. MQTT Version 5.0. OASIS Standard, 2019. Disponível em: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>. Acesso em: 09 jun. 2026.

TDK INVENSENSE. MPU-6000 and MPU-6050 Product Specification. Disponível em:

<https://invensense.tdk.com/products/motion-tracking/6-axis/mpu-6050/>. Acesso em: 09 jun. 2026.

SOCKET.IO. Socket.IO Documentation. Disponível em: <https://socket.io/docs/v4/>. Acesso em: 09 jun. 2026.

NODE.JS. Node.js Documentation. Disponível em: <https://nodejs.org/docs/latest/api/>. Acesso em: 09 jun. 2026.

MYSQL. MySQL 8.0 Reference Manual. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>.

Acesso em: 09 jun. 2026.

UNIVERSIDADE DO VALE DO PARAÍBA. Manual de Normatização de Trabalhos Acadêmicos. São José dos Campos: Univap, 2025.

Agradecimentos

Agradecemos ao Prof. Me. Hélio Esperidião pela orientação na disciplina Projetos em Engenharia da Computação II e à Universidade do Vale do Paraíba pelo ambiente acadêmico de desenvolvimento e demonstração do projeto.